Proyecto 01 - Sistemas Operativos

Estudiantes:

- Barrantes Jiménez Anthony
- Cabrera Tabash Samir

Link video: [Pendiente]

Tabla de Contenidos

1. Descripción General
2. Requisitos del Sistema
3. Instalación
4. Ejecución del Servidor
5. Configuración
6. Uso de la API
7. Testing
8. Arquitectura
9. Troubleshooting

Descripción General

Servidor HTTP/1.0 implementado en Rust que maneja múltiples tipos de operaciones concurrentes:

- **Operaciones básicas**: manipulación de texto, generación de números aleatorios, timestamps
- **Operaciones CPU-intensivas**: cálculos de números primos, factorización, cálculo de Pi, Mandelbrot, multiplicación de matrices
- **Operaciones I/O-intensivas**: ordenamiento de archivos, compresión, búsqueda de patrones, hashing de archivos
- **Sistema de Jobs**: gestión asíncrona de tareas largas con soporte para prioridades, cancelación y seguimiento de progreso

El servidor utiliza un sistema de workers especializados por tipo de tarea, colas de trabajo configurables y métricas detalladas de rendimiento.

Requisitos del Sistema

Software Necesario

- **Rust**: versión 1.70 o superior
  - Instalar desde: https://rustup.rs/
  - Verificar instalación: `rustc --version`
- **Cargo**: incluido con Rust

- Verificar: `cargo --version`

Dependencias del Proyecto

El proyecto utiliza las siguientes dependencias principales (gestionadas automáticamente por Cargo):

```
clap = "4.5"            # Parsing de argumentos CLI
serde = "1.0"           # Serialización JSON
sha2 = "0.10"           # Funciones de hash
flate2 = "1.0"          # Compresión GZIP
regex = "1.10"          # Expresiones regulares
chrono = "0.4"          # Manejo de fechas/tiempo
rand = "0.8"            # Generación de números aleatorios
```

Sistema Operativo

Probado en:

- Linux (Ubuntu 20.04+)
- macOS (Big Sur+)
- Windows 10/11 (con WSL recomendado)

Instalación

1. Clonar el Repositorio

```
git clone <URL_DEL_REPOSITORIO>
cd Proyecto-01
```

2. Compilar el Proyecto

```
cd Code
cargo build --release
```

La compilación puede tomar varios minutos la primera vez mientras se descargan y compilan las dependencias.

3. Verificar la Compilación

```
cargo check
```

Debe completar sin errores ni warnings.

Ejecución del Servidor

Comando Básico

Desde el directorio `Code/`:

```
cargo run -- --port=8080 --data-dir=data
```

Comando con Configuración Completa

Para aprovechar todas las capacidades del servidor:

```
cargo run -- \
  --port=8080 \
  --data-dir=data \
  --workers./sortfile=1 \
  --queue./sortfile=5 \
  --io-timeout=900000
```

Verificar que el Servidor Está Activo

```
curl http://localhost:8080/help
```

Debe retornar un JSON con la lista de endpoints disponibles.

Configuración

Parámetros de Línea de Comando

Parámetro	Descripción	Valor por Defecto	Ejemplo
`--port`	Puerto del servidor HTTP	8080	`--port=3000`
`--data-dir`	Directorio para archivos de datos	`./data`	`--data-dir=../data`
`--workers.<ruta>`	Número de workers por endpoint	4	`--workers./isprime=8`
`--queue.<ruta>`	Profundidad de cola por endpoint	100	`--queue./sortfile=50`
`--io-timeout`	Timeout para operaciones I/O (ms)	300000	`--io-timeout=600000`

Parámetro	Descripción	Valor por Defecto	Ejemplo
`--cpu-timeout`	Timeout para operaciones CPU (ms)	60000	`--cpu-timeout=120000`

Ejemplo de Configuración Optimizada

```
cargo run -- \
  --port=8080 \
  --data-dir=data \
  --workers./isprime=8 \
  --workers./sortfile=2 \
  --workers./compress=4 \
  --queue./sortfile=10 \
  --io-timeout=900000 \
  --cpu-timeout=120000
```

Variables de Entorno (Opcional)

Alternativamente, se pueden usar variables de entorno:

```
export HTTP_PORT=8080
export DATA_DIR=data
cargo run
```

Uso de la API

Colección de Postman

Para facilitar el testing, se proporciona una colección completa de Postman con todos los endpoints implementados:

Ubicación: `Proyecto-01/Postman/SO_Proyecto_HTTP_Server.postman_collection.json`

Importar en Postman:

1. Abrir Postman
2. Click en "Import"
3. Seleccionar el archivo JSON
4. La colección incluye:
    - Todos los endpoints básicos
    - Operaciones CPU-intensivas
    - Operaciones I/O-intensivas
    - Sistema de Jobs
    - Tests de errores y validación

Endpoints Principales

Información del Servidor

```
# Obtener ayuda
curl http://localhost:8080/help

# Estado del servidor
curl http://localhost:8080/status

# Métricas de rendimiento
curl http://localhost:8080/metrics
```

Operaciones Básicas

```
# Invertir texto
curl "http://localhost:8080/reverse?text=hello"

# Convertir a mayúsculas
curl "http://localhost:8080/toupper?text=hello"

# Generar hash SHA256
curl "http://localhost:8080/hash?text=hello"

# Fibonacci
curl "http://localhost:8080/fibonacci?num=20"

# Números aleatorios
curl "http://localhost:8080/random?count=10&min=1&max=100"
```

Operaciones CPU-Intensivas

```
# Test de primalidad
curl "http://localhost:8080/isprime?n=982451653&algo=mr"

# Factorización
curl "http://localhost:8080/factor?n=360"

# Cálculo de Pi
curl "http://localhost:8080/pi?digits=100&algo=spigot"

# Mandelbrot
curl "http://localhost:8080/mandelbrot?width=800&height=600&max_iter=1000"

# Multiplicación de matrices
curl "http://localhost:8080/matrixmul?size=128&seed=42"
```

Operaciones I/O-Intensivas

```
# Crear archivo
curl "http://localhost:8080/createfile?
name=test.txt&content=Hello&repeat=1000"

# Ordenar archivo
curl "http://localhost:8080/sortfile?name=test.txt&algo=merge"

# Contar palabras
curl "http://localhost:8080/wordcount?name=test.txt"

# Buscar patrón
curl "http://localhost:8080/grep?name=test.txt&pattern=Hello"

# Comprimir archivo
curl "http://localhost:8080/compress?name=test.txt&codec=gzip"

# Hash de archivo
curl "http://localhost:8080/hashfile?name=test.txt&algo=sha256"
```

Sistema de Jobs

```
# Enviar job
curl "http://localhost:8080/jobs/submit?
route=/sortfile&name=large.txt&algo=merge&prio=high"
# Respuesta: {"job_id":"0000019a378360fe-00000000","status":"queued"}

# Consultar estado
curl "http://localhost:8080/jobs/status?id=0000019a378360fe-00000000"

# Obtener resultado
curl "http://localhost:8080/jobs/result?id=0000019a378360fe-00000000"

# Cancelar job
curl "http://localhost:8080/jobs/cancel?id=0000019a378360fe-00000000"

# Listar todos los jobs
curl "http://localhost:8080/jobs/list"
```

Formato de Respuestas

Todas las respuestas son en formato JSON:

Respuesta Exitosa (200):

```
{
  "result": "...",
  "elapsed_ms": 123
}
```

Respuesta de Error (4xx/5xx):

```
{
  "error": "Descripción del error",
  "status": 400
}
```

Headers Importantes

Todas las respuestas incluyen:

- `X-Request-Id`: Identificador único de la petición
- `X-Worker-Pid`: PID del worker que procesó la petición (si aplica)
- `Content-Type`: `application/json`

Testing

Suite de Tests Completa

El proyecto incluye 40+ tests unitarios y de integración:

```
cd Code
cargo test
```

Output esperado:

```
running 40 tests
test algorithms::mandelbrot::tests::tiny_map ... ok
test algorithms::prime::tests::test_is_prime_basic ... ok
test algorithms::matrix_ops::tests::deterministic_hash ... ok
test handlers::basics::tests::test_fibonacci ... ok
test server::requests::tests::test_parse_simple_get ... ok
[...]

test result: ok. 40 passed; 0 failed; 0 ignored; 0 measured
```

Tests por Categoría

Tests Unitarios (40 tests):

- Algoritmos matemáticos (primos, Pi, Mandelbrot, matrices)
- Parsing HTTP
- Validación de parámetros
- Manejo de errores
- Utilidades (JSON, crypto, logging)

Tests de Integración (3 tests):

- Endpoints básicos
- Concurrencia
- Rendimiento comparativo

Ejecutar Tests Específicos

```
# Tests de algoritmos
cargo test algorithms

# Tests del servidor HTTP
cargo test server

# Tests de handlers
cargo test handlers

# Tests con output detallado
cargo test -- --nocapture

# Tests con un solo thread (para debugging)
cargo test -- --test-threads=1
```

Cobertura de Código

Para generar reporte de cobertura (requiere `tarpaulin`):

```
# Instalar tarpaulin
cargo install cargo-tarpaulin

# Generar reporte
cargo tarpaulin --out Html --output-dir coverage
```

El proyecto mantiene una cobertura superior al 90% según los requerimientos.

Arquitectura

Componentes Principales

```
http-server/
├── src/
│   ├── main.rs              # Punto de entrada
│   ├── lib.rs               # Librería principal
│   ├── server/              # Core HTTP
│   │   ├── http_server.rs   # Servidor TCP
│   │   ├── connection.rs    # Manejo de conexiones
│   │   ├── requests.rs      # Parser HTTP/1.0
│   │   ├── response.rs      # Constructor de respuestas
│   │   └── router.rs        # Sistema de routing
│   ├── workers/             # Sistema de workers
│   │   ├── worker_pool.rs   # Pool de threads
│   │   ├── worker_types.rs  # Tipos de threads
│   │   ├── task_queue.rs    # Calendarizador
│   │   └── worker_manager.rs
│   ├── jobs/                # Sistema de jobs
│   │   ├── job_manager.rs   # Gestor de jobs
│   │   ├── job_storage.rs   # Persistencia
│   │   ├── job_queue.rs.    # Cola de trabajos
│   │   ├── job_scheduler.rs # Calendarizacion de trabajos
│   │   └── job_types.rs     # Tipos y estados
│   ├── handlers/            # Implementación de endpoints
│   │   ├── basics.rs        # Endpoints básicos
│   │   ├── cpu_intensive.rs # Operaciones CPU
│   │   ├── handler_traits.rs# Funcionalidades comunes
│   │   ├── io_intensive.rs  # Operaciones I/O
│   │   ├── job_endpoints.rs # API de jobs
│   │   └── metrics.rs       # Métricas
│   ├── algorithms/          # Algoritmos implementados
│   ├── io_operations/       # Operaciones de archivos
│   ├── utils/               # Utilidades
│   └── error/               # Manejo de errores
└── tests/                   # Tests de integración
```

Flujo de Ejecución

1. **Cliente** envía request HTTP/1.0
2. **Servidor TCP** acepta conexión
3. **Parser** extrae método, ruta y parámetros
4. **Router** determina el handler apropiado
5. **Handler** valida parámetros y crea tarea
6. **Worker Manager** asigna tarea al pool correspondiente
7. **Worker** ejecuta la tarea
8. **Handler** construye respuesta JSON
9. **Servidor** envía respuesta al cliente

Modelo de Concurrencia

- **Thread pool por tipo de tarea**: CPU-bound, I/O-bound, básicas
- **Colas de trabajo**: FIFO con soporte para prioridades

- **Jobs asíncronos**: Para tareas largas con polling de estado
- **Thread-safe**: Uso de `Arc<Mutex<>>` y channels para sincronización

Troubleshooting

Problemas Comunes

El servidor no inicia

Error: `Address already in use`

Solución: El puerto 8080 está ocupado. Usar otro puerto:

```
cargo run -- --port=8081
```

Error: `No such file or directory: data/`

Solución: Crear el directorio de datos:

```
mkdir -p data
```

Errores de compilación

Error: `failed to resolve: use of undeclared crate`

Solución: Limpiar y recompilar:

```
cargo clean
cargo build
```

Timeout en operaciones I/O

Error: `{"error":"Operation timed out"}`

Solución: Aumentar el timeout:

```
cargo run -- --io-timeout=1800000  # 30 minutos
```

Archivos no encontrados

Error: `{"error":"File not found: test.txt"}`

Verificar:

- El archivo debe estar en el directorio especificado por `--data-dir`
- Por defecto: `Code/data/`
- Usar rutas relativas sin `../`

Logs de Debugging

Activar logs detallados:

```
RUST_LOG=debug cargo run -- --port=8080
```

Niveles disponibles:

- `error`: Solo errores críticos
- `warn`: Advertencias
- `info`: Información general (default)
- `debug`: Información detallada
- `trace`: Información muy detallada

Verificar Estado del Servidor

```
# Estado general
curl http://localhost:8080/status

# Métricas detalladas
curl http://localhost:8080/metrics

# Workers activos y colas
curl http://localhost:8080/metrics | jq '.workers'
```

Performance Issues

Si el servidor responde lento:

1. **Aumentar workers**:

```
cargo run -- --workers./isprime=16
```

2. **Aumentar profundidad de colas**:

```
cargo run -- --queue./sortfile=200
```

3. **Usar modo release** (más rápido):

```
cargo build --release
./target/release/http-server --port=8080
```

Contacto y Soporte

Para problemas, preguntas o sugerencias:

- **Autores**: Anthony Barrantes, Samir Cabrera
- **Curso**: Principios de Sistemas Operativos
- **Institución**: TEC - Sede Central Cartago

Licencia

Este proyecto es desarrollado con fines académicos para el curso de Sistemas Operativos.